

Lecture 3: Optimization with Gradient Descent

Machine learning models are trained by solving an optimization problem: we search for the parameters that make the model's predictions match the training data as closely as possible.

A loss function is a function that measures how far a model's predictions are from the true targets. Common examples are the squared error for regression and the cross-entropy loss for classification.

Empirical risk is the average loss over all examples in the training set. Training a model means minimizing the empirical risk with respect to the model parameters, usually written as θ .

For most models there is no closed-form solution, so we use iterative optimization methods. This lecture covers gradient descent and the variants that are used to train modern neural networks.

The Gradient Descent Algorithm

The gradient is the vector of partial derivatives of the loss with respect to each parameter. It points in the direction of steepest increase of the loss.

Gradient descent is an iterative optimization algorithm that updates the parameters in the direction of the negative gradient. Each iteration applies the update rule: $\theta_{\text{new}} = \theta - \eta * \text{grad } L(\theta)$.

The learning rate is a hyperparameter that controls the size of each update step. It is written as η and is usually between 0.0001 and 0.1.

The algorithm repeats the update until the loss stops decreasing, a maximum number of iterations is reached, or the norm of the gradient falls below a small tolerance.

Choosing the Learning Rate

If the learning rate is too large, the updates overshoot the minimum: the loss oscillates or even diverges to infinity. If the learning rate is too small, training converges very slowly and may stall on plateaus.

A learning rate schedule changes the learning rate during training. Popular schedules include step decay, exponential decay and cosine annealing, which smoothly lowers the rate following a cosine curve.

Warmup is a technique that starts training with a small learning rate and increases it gradually over the first few hundred steps. Warmup stabilizes the early phase of training for large models.

A practical way to pick an initial value is a learning rate range test: train briefly while increasing the rate exponentially and choose a value slightly below the point where the loss starts to explode.

Stochastic and Mini-batch Gradient Descent

Batch gradient descent computes the exact gradient over the entire training set, which is expensive when the dataset contains millions of examples.

Stochastic gradient descent (SGD) estimates the gradient using a single randomly chosen training example. Each step is cheap but noisy.

Mini-batch gradient descent computes the gradient on a small random subset of examples called a batch. Typical batch sizes range from 32 to 256 and make good use of GPU parallelism.

An epoch is one full pass over the training dataset. The noise in stochastic updates can help the optimizer escape saddle points and shallow local minima.

Momentum and Adaptive Methods

Momentum is a method that accumulates an exponentially decaying average of past gradients to accelerate descent. It dampens oscillations in steep directions and speeds up progress along shallow, consistent directions.

RMSProp divides the learning rate for each parameter by a running average of recent squared gradients, so parameters with large gradients take smaller steps.

Adam is an adaptive optimizer that combines momentum with per-parameter learning rates based on estimates of the first and second moments of the gradients. Its default hyperparameters are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 1e-8$.

AdamW decouples weight decay from the adaptive gradient update and is the default optimizer for training transformer models.

Convergence and Practical Tips

For convex loss functions, gradient descent with a suitable learning rate is guaranteed to converge to the global minimum. Neural network losses are non-convex and contain many local minima and saddle points.

Feature scaling speeds up convergence: standardizing each input feature to zero mean and unit variance makes the loss surface more round, so gradient steps point closer to the minimum.

Gradient clipping is a technique that rescales gradients whose norm exceeds a threshold. It prevents exploding gradients in recurrent networks and very deep models.

Always monitor both training loss and validation loss. A training loss that decreases while the validation loss increases is a sign of overfitting, which is covered in the regularization notes.